

prime_scoring.py

"""

prime_scoring.py

=====

PrIME (Pain-generator, Impression, and Musculoskeletal Evaluation) Scoring Engine

Published rubric: Appendix Table 1 of the PrIME benchmark manuscript.

FIELD WEIGHTS

Pain Generator (PG) : 35%

Side / Laterality : 25%

Spinal Level : 20%

Nerve Root (NR) : 10%

Disc / Bone (DB) : 10%

SCORING TIERS (0–4) — applied per field

Score 4 : Exact match to expert-consensus label

Score 3 : Exact match to one expert (non-consensus) OR adjacent (field-specific)

Score 2 : Clinically adjacent / hedged / 2-step away

Score 1 : Related but wrong

Score 0 : Hallucinated, anatomically impossible, or vocabulary-violating

CONSENSUS vs NON-CONSENSUS LOGIC

When BOTH experts agree (consensus), adjacency penalties are applied from the shared consensus label.

When experts disagree (non-consensus), a match to EITHER expert yields

Score 3; exact adjacency relative to either expert drives Scores 1-2.

USAGE

```
from prime_scoring import PRIMEScorer, build_gt
```

```
gt = build_gt(yc_pg='stenosis: subarticular', fran_pg='stenosis: foraminal',  
yc_level='l4-5', fran_level='l4-5',  
yc_side='right', fran_side='right',  
yc_nerve='l5', fran_nerve='l5',  
yc_db='disc', fran_db='disc')
```

```
scorer = PRIMEScorer()  
result = scorer.score_case(  
llm_pg='stenosis: subarticular',  
llm_level='l4-5',  
llm_side='right',  
llm_nerve='l5',  
llm_db='disc',  
gt=gt  
)  
print(result)  
# {'pg': 4, 'level': 4, 'side': 4, 'nerve': 4, 'db': 4,  
# 'composite_raw': 4.0, 'composite_pct': 100.0,  
# 'consensus': {'pg': True, 'level': True, 'side': True,  
# 'nerve': True, 'db': True}}  
.....
```

```
from __future__ import annotations
import re
from dataclasses import dataclass, field
from typing import Optional
```

```
#
```

```
# VOCABULARY NORMALIZATION TABLES
```

```
#
```

```
# Canonical pain-generator labels (closed vocabulary)
```

```
PAIN_GEN_ALIASES: dict[str, str] = {
```

```
# — Central / Canal stenosis —————
```

```
"stenosis: canal": "stenosis: canal",
```

```
"stenosis: central canal": "stenosis: canal",
```

```
"central canal stenosis": "stenosis: canal",
```

```
"central stenosis": "stenosis: canal",
```

```
"spinal stenosis": "stenosis: canal",
```

```
"lumbar spinal stenosis": "stenosis: canal",
```

```
"lumbar stenosis": "stenosis: canal",
```

```
# — Subarticular / Lateral recess —————
```

```
"stenosis: subarticular": "stenosis: subarticular",
```

```
"stenosis: subarticular zone": "stenosis: subarticular",
```

```
"subarticular stenosis": "stenosis: subarticular",
```

```
"lateral recess stenosis": "stenosis: subarticular",
```

— Foraminal stenosis —————

"stenosis: foraminal": "stenosis: foraminal",

"stenosis: foramen": "stenosis: foraminal",

"foraminal stenosis": "stenosis: foraminal",

"foraminal narrowing": "stenosis: foraminal",

— Disc herniation / Annular tear —————

"disc hrn/annular tear+no stenosis": "disc hrn/annular tear",

"disc hrn/annular tear": "disc hrn/annular tear",

"disk herniation, annular tear no stenosis": "disc hrn/annular tear",

"disc herniation, annular tear no stenosis": "disc hrn/annular tear",

"disc herniation": "disc hrn/annular tear",

"disk herniation": "disc hrn/annular tear",

"herniated disc": "disc hrn/annular tear",

"herniated disk": "disc hrn/annular tear",

"disc herniation or stenosis": "disc hrn/annular tear",

"annular tear": "disc hrn/annular tear",

— DDD with bone marrow edema —————

"ddd w/ bone marrow edema": "ddd w/ bone marrow edema",

"degenerative disc disease with bone marrow edema": "ddd w/ bone marrow edema",

"ddd with bone marrow edema": "ddd w/ bone marrow edema",

— Facet arthropathy —————

"facet arthropathy": "facet arthropathy",

"facet arthropathy & instability": "facet arthropathy",

"facet joint arthropathy": "facet arthropathy",

"facet joint degeneration": "facet arthropathy",

"facet joint disease": "facet arthropathy",

```
"facet joint pain": "facet arthropathy",  
"facet joint": "facet arthropathy",  
"z-joint": "facet arthropathy",  
"zygapophyseal joint": "facet arthropathy",  
"z-joint disease": "facet arthropathy",  
"degenerative facet arthropathy": "facet arthropathy",  
"degenerative facet disease": "facet arthropathy",
```

```
# — Spondylolysis
```

```
"spondylolysis": "spondylolysis",  
"pars defect": "spondylolysis",  
"pars interarticularis defect": "spondylolysis",  
}
```

```
# Disease family groupings for cross-subtype scoring
```

```
PAIN_GEN_FAMILIES: dict[str, str] = {  
"stenosis: canal": "stenosis",  
"stenosis: subarticular": "stenosis",  
"stenosis: foraminal": "stenosis",  
"disc hrn/annular tear": "disc",  
"ddd w/ bone marrow edema": "disc",  
"facet arthropathy": "bone_joint",  
"spondylolysis": "bone_joint",  
}
```

```
# Side / laterality canonical labels
```

```
SIDE_ALIASES: dict[str, str] = {
```

```

"right": "right",
"left": "left",
"bilateral": "midline/bilateral",
"midline": "midline/bilateral",
"midline/bilateral": "midline/bilateral",
"midline or left": "midline/bilateral",
"midline or right": "midline/bilateral",
"both": "midline/bilateral",
"b/l": "midline/bilateral",
"righth": "right", # common typo
}

```

Lumbar level canonical labels (ordered cephalad → caudad)

```
LEVEL_ORDER = ["l1-2", "l2-3", "l3-4", "l4-5", "l5-s1"]
```

```

LEVEL_ALIASES: dict[str, str] = {
    "l1-2": "l1-2", "l1-l2": "l1-2", "l1/l2": "l1-2",
    "l2-3": "l2-3", "l2-l3": "l2-3", "l2/l3": "l2-3",
    "l3-4": "l3-4", "l3-l4": "l3-4", "l3/l4": "l3-4",
    "l4-5": "l4-5", "l4-l5": "l4-5", "l4/l5": "l4-5",
    "l5-s1": "l5-s1", "l5-s2": "l5-s1", "l5/s1": "l5-s1",
    "l5 (if pars defect)": "l5-s1",
    "ns": "ns",
    "not specified": "ns",
}

```

Multi-level entries → primary level (first listed)

LEVEL_MULTI: dict[str, str] = {

"l4-5 or l5-s1": "l4-5",

"l4-l5 or l5-s1": "l4-5",

"l3-4 or l4-5": "l3-4",

"l3-l4 or l4-l5": "l3-4",

"l2-3 or l3-4": "l2-3",

}

Nerve root canonical labels (ordered cephalad → caudad)

NERVE_ORDER = ["l1", "l2", "l3", "l4", "l5", "s1", "s2"]

NERVE_ALIASES: dict[str, str] = {

"l1": "l1", "l2": "l2", "l3": "l3",

"l4": "l4", "l5": "l5",

"s1": "s1", "s2": "s2",

"ns": "ns", "n/a": "ns", "none": "ns",

"not applicable": "ns", "not specified": "ns",

"unknown": "ns", "no": "ns", "na": "ns",

}

NERVE_MULTI: dict[str, str] = {

"l5 and s1": "l5",

"l4 or l5": "l4",

"l5 or s1": "l5",

"l3 or l4": "l3",

```
"l4 or l5 or s1": "l4",  
"l5 or s1 or s2": "l5",  
}
```

```
# Disc / bone canonical labels
```

```
DISC_BONE_ALIASES: dict[str, str] = {  
    "disc": "disc",  
    "bone": "bone",  
    "disc or bone": "disc or bone",  
    "disc/bone": "disc or bone",  
    "disc disease": "disc",  
    "disc herniation": "disc",  
    "degenerative disc disease": "disc",  
    "bone disease": "bone",  
    "facet arthropathy": "bone",  
    "facet joint disease": "bone",  
    "z-joint disease": "bone",  
    "degenerative joint disease": "bone",  
    "ns": "ns",  
    "not specified": "ns",  
}
```

```
#
```

```
# NORMALIZATION HELPERS
```


#

```
def _clean(val) -> Optional[str]:
```

```
    """Return lowercase-stripped string, or None for missing/NaN."""
```

```
    if val is None:
```

```
        return None
```

```
    if isinstance(val, float):
```

```
        import math
```

```
        if math.isnan(val):
```

```
            return None
```

```
        s = str(val).strip().lower()
```

```
        return s if s else None
```

```
def normalize_pg(val) -> Optional[str]:
```

```
    """Normalize a pain-generator string to canonical vocabulary, or None."""
```

```
    s = _clean(val)
```

```
    if s is None:
```

```
        return None
```

```
    return PAIN_GEN_ALIASES.get(s)
```

```
def normalize_side(val) -> Optional[str]:
```

```
    s = _clean(val)
```

```
    if s is None:
```

```
return None
```

```
return SIDE_ALIASES.get(s)
```

```
def normalize_level(val) -> Optional[str]:
```

```
    s = _clean(val)
```

```
    if s is None:
```

```
        return None
```

```
    # Check multi-level shorthands first
```

```
    if s in LEVEL_MULTI:
```

```
        return LEVEL_MULTI[s]
```

```
    return LEVEL_ALIASES.get(s)
```

```
def normalize_nerve(val) -> str:
```

```
    """Returns canonical nerve string; defaults to 'ns' when blank."""
```

```
    s = _clean(val)
```

```
    if s is None:
```

```
        return "ns"
```

```
    if s in NERVE_MULTI:
```

```
        return NERVE_MULTI[s]
```

```
    return NERVE_ALIASES.get(s, "ns")
```

```
def normalize_db(val) -> Optional[str]:
```

```
    s = _clean(val)
```

```
if s is None:
    return None

# Protect "disc or bone" before substring checks

if s in DISC_BONE_ALIASES:
    return DISC_BONE_ALIASES[s]

if "bone" in s and "disc" not in s:
    return "bone"

if "disc" in s and "bone" not in s:
    return "disc"

return None
```

```
#
```

```
# GROUND TRUTH STRUCTURE
```

```
#
```

```
@dataclass
```

```
class GroundTruth:
```

```
    """
```

```
    Holds normalized labels for both experts (and optional Pooled adjudication).
```

```
    consensus[field] == True when expert1 == expert2 after normalization.
```

```
    """
```

```
    # Primary (normalized) labels per rater
```

```
    yc_pg: Optional[str] = None
```

```
    fran_pg: Optional[str] = None
```

yc_level: Optional[str] = None
fran_level: Optional[str] = None
yc_side: Optional[str] = None
fran_side: Optional[str] = None
yc_nerve: str = "ns"
fran_nerve: str = "ns"
yc_db: Optional[str] = None
fran_db: Optional[str] = None

Optional additional labels (second pain generator, second level)

yc_pg2: Optional[str] = None
fran_pg2: Optional[str] = None
yc_level2: Optional[str] = None
fran_level2: Optional[str] = None
yc_nerve2: str = "ns"
fran_nerve2: str = "ns"

Optional pooled adjudication

pooled_pg: Optional[str] = None
pooled_level: Optional[str] = None
pooled_side: Optional[str] = None
pooled_nerve: str = "ns"
pooled_db: Optional[str] = None

Computed on post_init

consensus: dict = field(default_factory=dict)

```
def __post_init__(self):
    self.consensus = {
        "pg": self.yc_pg == self.fran_pg and self.yc_pg is not None,
        "level": self.yc_level == self.fran_level and self.yc_level is not None,
        "side": self.yc_side == self.fran_side and self.yc_side is not None,
        "nerve": self.yc_nerve == self.fran_nerve,
        "db": self.yc_db == self.fran_db and self.yc_db is not None,
    }
```

```
def pg_labels(self) -> list[str]:
    """All valid PG labels (primary + additional, both raters, pooled)."""
    return [v for v in [self.yc_pg, self.fran_pg, self.yc_pg2,
        self.fran_pg2, self.pooled_pg] if v]
```

```
def level_labels(self) -> list[str]:
    return [v for v in [self.yc_level, self.fran_level, self.yc_level2,
        self.fran_level2, self.pooled_level] if v and v != "ns"]
```

```
def side_labels(self) -> list[str]:
    return [v for v in [self.yc_side, self.fran_side, self.pooled_side] if v]
```

```
def nerve_labels(self) -> list[str]:
    return [v for v in [self.yc_nerve, self.fran_nerve, self.yc_nerve2,
        self.fran_nerve2, self.pooled_nerve]
        if v and v != "ns"]
```

```
def db_labels(self) -> list[str]:  
    return [v for v in [self.yc_db, self.fran_db, self.pooled_db] if v]
```

```
@property
```

```
def nerve_is_ns(self) -> bool:
```

```
    """True if ALL expert labels are 'ns' (no specific nerve root expected)."""
```

```
    return len(self.nerve_labels()) == 0
```

```
def build_gt(
```

```
    yc_pg=None, fran_pg=None,
```

```
    yc_level=None, fran_level=None,
```

```
    yc_side=None, fran_side=None,
```

```
    yc_nerve=None, fran_nerve=None,
```

```
    yc_db=None, fran_db=None,
```

```
    yc_pg2=None, fran_pg2=None,
```

```
    yc_level2=None, fran_level2=None,
```

```
    yc_nerve2=None, fran_nerve2=None,
```

```
    pooled_pg=None, pooled_level=None, pooled_side=None,
```

```
    pooled_nerve=None, pooled_db=None,
```

```
) -> GroundTruth:
```

```
    """
```

Convenience constructor that normalizes raw strings before building GroundTruth.

Pass raw (un-normalized) strings from the spreadsheet.

```
    """
```

```
return GroundTruth(  
    yc_pg=normalize_pg(yc_pg),  
    fran_pg=normalize_pg(fran_pg),  
    yc_level=normalize_level(yc_level),  
    fran_level=normalize_level(fran_level),  
    yc_side=normalize_side(yc_side),  
    fran_side=normalize_side(fran_side),  
    yc_nerve=normalize_nerve(yc_nerve),  
    fran_nerve=normalize_nerve(fran_nerve),  
    yc_db=normalize_db(yc_db),  
    fran_db=normalize_db(fran_db),  
    yc_pg2=normalize_pg(yc_pg2),  
    fran_pg2=normalize_pg(fran_pg2),  
    yc_level2=normalize_level(yc_level2),  
    fran_level2=normalize_level(fran_level2),  
    yc_nerve2=normalize_nerve(yc_nerve2),  
    fran_nerve2=normalize_nerve(fran_nerve2),  
    pooled_pg=normalize_pg(pooled_pg),  
    pooled_level=normalize_level(pooled_level),  
    pooled_side=normalize_side(pooled_side),  
    pooled_nerve=normalize_nerve(pooled_nerve),  
    pooled_db=normalize_db(pooled_db),  
)
```

#

INDIVIDUAL FIELD SCORERS

(each returns an integer 0–4 and a short rationale string)

#

```
def _level_dist(a: str, b: str) -> Optional[int]:
```

```
    """Ordinal distance between two lumbar levels; None if either is invalid."""
```

```
    if a not in LEVEL_ORDER or b not in LEVEL_ORDER:
```

```
        return None
```

```
    return abs(LEVEL_ORDER.index(a) - LEVEL_ORDER.index(b))
```

```
def _nerve_dist(a: str, b: str) -> Optional[int]:
```

```
    """Ordinal distance between two nerve root labels; None if either is invalid."""
```

```
    if a not in NERVE_ORDER or b not in NERVE_ORDER:
```

```
        return None
```

```
    return abs(NERVE_ORDER.index(a) - NERVE_ORDER.index(b))
```

```
def score_pain_gen(llm_val, gt: GroundTruth) -> tuple[int, str]:
```

```
    """
```

Score 4: LLM matches consensus (both experts agree and LLM = their label).

Score 3: LLM matches one expert (non-consensus) OR a Pooled adjudication label.

Score 2: LLM is in the same disease family but wrong subtype

(e.g., foraminal vs. subarticular — both stenosis).

Score 1: LLM names a real spinal structural finding in a different family.

Score 0: Hallucinated, vocabulary-violating, or unparseable.

```
"""
```

```
llm = normalize_pg(llm_val)
```

```
if llm is None:
```

```
    return 0, f"unparseable PG: {repr(llm_val)!r}"
```

```
all_labels = gt.pg_labels()
```

```
if not all_labels:
```

```
    return 0, "no expert PG labels available"
```

```
# Score 4: consensus exact match
```

```
if gt.consensus["pg"] and llm == gt.yc_pg:
```

```
    return 4, f"consensus exact match: {llm}"
```

```
# Score 3: matches either expert or pooled (non-consensus or pooled adj.)
```

```
if llm in all_labels:
```

```
    return 3, f"matches one expert/pooled: {llm}"
```

```
# Score 2 / 1: family-based partial credit
```

```
llm_fam = PAIN_GEN_FAMILIES.get(llm)
```

```
gt_fams = {PAIN_GEN_FAMILIES.get(lbl) for lbl in all_labels if lbl in PAIN_GEN_FAMILIES}
```

```
gt_fams.discard(None)
```

```
if llm_fam and llm_fam in gt_fams:
```

```
    return 2, f"same disease family ({llm_fam}) but wrong subtype: {llm}"
```

```

# Score 1: it's a known structural finding, just the wrong family
if llm in PAIN_GEN_FAMILIES:
    return 1, f"different disease family ({llm_fam} vs {gt_fams}): {llm}"

return 0, f"vocabulary-violating PG: {llm}"

```

```

def score_side(llm_val, gt: GroundTruth) -> tuple[int, str]:
    """
    Score 4: Exact match to consensus label.
    Score 3: Matches one expert (non-consensus) or pooled.
    Score 2: LLM hedges to bilateral when experts specified unilateral.
    Score 1: Opposite specific laterality (right vs. left).
    Score 0: Anatomically invalid or unparseable.
    """

    llm = normalize_side(llm_val)

    if llm is None:
        return 0, f"unparseable side: {repr(llm_val)!r}"

    all_labels = gt.side_labels()

    if not all_labels:
        return 0, "no expert side labels available"

    # Score 4: consensus
    if gt.consensus["side"] and llm == gt.yc_side:

```

```
return 4, f"consensus exact match: {llm}"
```

```
# Score 3: matches one expert or pooled
```

```
if llm in all_labels:
```

```
return 3, f"matches one expert/pooled: {llm}"
```

```
# Score 2: overly broad hedging (bilateral when unilateral expected)
```

```
if llm == "midline/bilateral" and any(l in ("right", "left") for l in all_labels):
```

```
return 2, "LLM hedged bilateral; expert specified unilateral"
```

```
# Score 1: opposite unilateral
```

```
unilateral_map = {"right": "left", "left": "right"}
```

```
if llm in unilateral_map and unilateral_map[llm] in all_labels:
```

```
return 1, f"opposite laterality: LLM {llm}, expert {unilateral_map[llm]}"
```

```
return 0, f"anatomically invalid side: {llm}"
```

```
def score_level(llm_val, gt: GroundTruth) -> tuple[int, str]:
```

```
"""
```

```
Score 4: Exact match to consensus level.
```

```
Score 3: Matches one expert in non-consensus OR is  $\pm 1$  level from consensus.
```

```
Score 2: Adjacent ( $\pm 1$ ) to one expert in non-consensus OR  $\pm 2$  from consensus.
```

```
Score 1:  $\pm 2$  from one non-consensus expert OR  $\pm 3$  from consensus.
```

```
Score 0: Cervical/thoracic level, NS when expert is specific, or unparseable.
```

```
"""
```

```

llm = normalize_level(llm_val)

if llm is None:
    return 0, f"unparseable level: {repr(llm_val)!r}"

if llm == "ns":
    return 0, "model returned NS; expert specified a level"

all_levels = gt.level_labels()
if not all_levels:
    # Expert also said NS — treat as match
    return 4, "both model and experts: no specific level"

if llm not in LEVEL_ORDER:
    return 0, f"non-lumbar or invalid level: {llm}"

# Score 4: consensus exact
if gt.consensus["level"] and llm == gt.yc_level:
    return 4, f"consensus exact match: {llm}"

# Score 3: non-consensus exact OR  $\pm 1$  from consensus
if llm in all_levels:
    return 3, f"matches one expert: {llm}"

if gt.consensus["level"]:
    d = _level_dist(llm, gt.yc_level)
    if d == 1:

```

```

return 3, f"adjacent ( $\pm 1$ ) to consensus {gt.yc_level}: {llm}"

if d == 2:
    return 2, f" $\pm 2$  levels from consensus {gt.yc_level}: {llm}"

if d is not None:
    return 1, f" $\pm {d}$  levels from consensus {gt.yc_level}: {llm}"
else:
    # Non-consensus: score against best (closest) expert
    dists = [_level_dist(llm, lbl) for lbl in all_levels if lbl in LEVEL_ORDER]
    dists = [d for d in dists if d is not None]

    if not dists:
        return 0, "no valid level distances computable"

    best = min(dists)

    if best == 1:
        return 2, f" $\pm 1$  level from one expert (non-consensus): {llm}"

    if best == 2:
        return 1, f" $\pm 2$  levels from one expert (non-consensus): {llm}"

    return 0, f" $\geq 3$  levels from any expert: {llm}"

return 0, f"no match: {llm}"

```

```

def score_nerve(llm_val, gt: GroundTruth) -> tuple[int, str]:

```

```

    """

```

Score 4: Exact match to consensus nerve OR both experts and model say NS.

Score 3: Matches one expert in non-consensus OR ± 1 nerve from consensus.

Score 2: ± 1 nerve from one expert in non-consensus OR ± 2 from consensus.

Also: model names a nerve when experts said NS (partial — nerve implied by level but not specified).

Score 1: ± 2 from one non-consensus expert OR ± 3 from consensus.

Score 0: Thoracic/cervical nerve, nonsensical value.

"""

```
llm = normalize_nerve(llm_val)
```

```
# Both NS → Score 4
```

```
if gt.nerve_is_ns and llm == "ns":
```

```
    return 4, "both model and experts: no nerve root specified (NS)"
```

```
# Model named a nerve when experts said NS → Score 2
```

```
if gt.nerve_is_ns and llm != "ns":
```

```
    return 2, f"model named nerve {llm} but experts said NS"
```

```
# Experts specified a nerve, model said NS
```

```
if not gt.nerve_is_ns and llm == "ns":
```

```
    return 0, "model returned NS; expert specified a nerve root"
```

```
if llm not in NERVE_ORDER:
```

```
    return 0, f"non-lumbar/invalid nerve: {llm}"
```

```
all_nerves = gt.nerve_labels()
```

```
# Score 4: consensus exact
```

```
if gt.consensus["nerve"] and llm == gt.yc_nerve:
```

```

return 4, f"consensus exact match: {llm}"

# Score 3: non-consensus exact OR  $\pm 1$  from consensus
if llm in all_nerves:
    return 3, f"matches one expert: {llm}"

if gt.consensus["nerve"]:
    d = _nerve_dist(llm, gt.yc_nerve)
    if d == 1:
        return 3, f"adjacent ( $\pm 1$ ) to consensus {gt.yc_nerve}: {llm}"
    if d == 2:
        return 2, f" $\pm 2$  from consensus {gt.yc_nerve}: {llm}"
    if d is not None:
        return 1, f" $\pm {d}$  from consensus {gt.yc_nerve}: {llm}"
    else:
        dists = [_nerve_dist(llm, n) for n in all_nerves if n in NERVE_ORDER]
        dists = [d for d in dists if d is not None]
        if not dists:
            return 0, "no valid nerve distances"
        best = min(dists)
        if best == 1:
            return 2, f" $\pm 1$  nerve from one expert (non-consensus): {llm}"
        if best == 2:
            return 1, f" $\pm 2$  nerves from one expert (non-consensus): {llm}"
        return 0, f" $\geq 3$  nerves from any expert: {llm}"

```

```
return 0, f"no match: {llm}"
```

```
def score_disc_bone(llm_val, gt: GroundTruth) -> tuple[int, str]:
```

```
    """
```

Score 4: Exact match to consensus label (disc, bone, or disc or bone).

Score 3: Matches one expert (non-consensus) or pooled.

Also: specific answer (disc or bone) when expert consensus is "disc or bone".

Score 2: LLM hedges "disc or bone" when expert was specific.

Score 1: Wrong specific label (disc vs. bone).

Score 0: Unmatched, invalid, or vocabulary-violating.

```
    """
```

```
    llm = normalize_db(llm_val)
```

```
    if llm is None:
```

```
        return 0, f"unparseable disc/bone: {repr(llm_val)!r}"
```

```
    all_labels = gt.db_labels()
```

```
    if not all_labels:
```

```
        return 0, "no expert disc/bone labels available"
```

```
    # Score 4: consensus exact
```

```
    if gt.consensus["db"] and llm == gt.yc_db:
```

```
        return 4, f"consensus exact match: {llm}"
```

```
    # Score 4: expert consensus is "disc or bone", any specific answer is fine
```

```
    if gt.consensus["db"] and gt.yc_db == "disc or bone" and llm in ("disc", "bone"):
```



```
return 4, f"expert hedged 'disc or bone'; specific {llm} accepted"
```

```
# Score 3: matches one expert or pooled
```

```
if llm in all_labels:
```

```
return 3, f"matches one expert/pooled: {llm}"
```

```
# Score 3: expert set contains "disc or bone"; specific answer still counts
```

```
if "disc or bone" in all_labels and llm in ("disc", "bone"):
```

```
return 3, f"one expert hedged; specific {llm} matches partial"
```

```
# Score 2: LLM hedges when experts were specific
```

```
if llm == "disc or bone" and all(l in ("disc", "bone") for l in all_labels):
```

```
return 2, "LLM hedged 'disc or bone'; expert gave specific answer"
```

```
# Score 1: wrong specific label
```

```
if llm in ("disc", "bone") and any(l in ("disc", "bone") for l in all_labels):
```

```
return 1, f"wrong designation: LLM {llm}, expert {all_labels}"
```

```
return 0, f"vocabulary-violating disc/bone: {llm}"
```

```
#
```

```
# COMPOSITE SCORER
```

```
#
```

```
FIELD_WEIGHTS = {  
    "pg": 0.35,  
    "side": 0.25,  
    "level": 0.20,  
    "nerve": 0.10,  
    "db": 0.10,  
}
```

```
@dataclass  
class PRIMEResult:  
    pg: int  
    side: int  
    level: int  
    nerve: int  
    db: int  
    pg_rationale: str = ""  
    side_rationale: str = ""  
    level_rationale: str = ""  
    nerve_rationale: str = ""  
    db_rationale: str = ""  
    composite_raw: float = 0.0 # 0–4 scale  
    composite_pct: float = 0.0 # 0–100 scale  
    consensus: dict = field(default_factory=dict)  
  
    def __post_init__(self):
```

```
self.composite_raw = round(
self.pg * FIELD_WEIGHTS["pg"] +
self.side * FIELD_WEIGHTS["side"] +
self.level * FIELD_WEIGHTS["level"] +
self.nerve * FIELD_WEIGHTS["nerve"] +
self.db * FIELD_WEIGHTS["db"],
4,
)
self.composite_pct = round(self.composite_raw / 4 * 100, 2)
```

```
def to_dict(self) -> dict:
return {
"PrIME_PG": self.pg,
"PrIME_Side": self.side,
"PrIME_Level": self.level,
"PrIME_Nerve": self.nerve,
"PrIME_DB": self.db,
"PrIME_Composite_raw": self.composite_raw,
"PrIME_Composite_pct": self.composite_pct,
"PG_rationale": self.pg_rationale,
"Side_rationale": self.side_rationale,
"Level_rationale": self.level_rationale,
"Nerve_rationale": self.nerve_rationale,
"DB_rationale": self.db_rationale,
"consensus_pg": self.consensus.get("pg"),
"consensus_side": self.consensus.get("side"),
```

```
"consensus_level": self.consensus.get("level"),
"consensus_nerve": self.consensus.get("nerve"),
"consensus_db": self.consensus.get("db"),
}
```

```
class PRIMEScorer:
```

```
"""
```

Main entry point. Instantiate once; call score_case() per LLM output row.

Example

```
scorer = PRIMEScorer()
```

```
gt = build_gt(
yc_pg='Stenosis: Subarticular',
fran_pg='Stenosis: Subarticular',
yc_level='L4-5', fran_level='L4-5',
yc_side='Right', fran_side='Right',
yc_nerve='L5', fran_nerve='L5',
yc_db='Disc', fran_db='Disc',
)
```

```
result = scorer.score_case(
llm_pg='stenosis: subarticular',
llm_level='L4-5',
```

```

llm_side='right',
llm_nerve='L5',
llm_db='disc',
gt=gt,
)
# result.composite_pct → 100.0
"""

```

```

def score_case(
self,
llm_pg,
llm_level,
llm_side,
llm_nerve,
llm_db,
gt: GroundTruth,
) -> PrIMERResult:
    pg, pg_r = score_pain_gen(llm_pg, gt)
    side, side_r = score_side(llm_side, gt)
    level, level_r = score_level(llm_level, gt)
    nerve, nerve_r = score_nerve(llm_nerve, gt)
    db, db_r = score_disc_bone(llm_db, gt)

    return PrIMERResult(
        pg=pg, side=side, level=level, nerve=nerve, db=db,
        pg_rationale=pg_r, side_rationale=side_r,

```

```
level_rationale=level_r, nerve_rationale=nerve_r,  
db_rationale=db_r,  
consensus=gt.consensus,  
)
```

```
def score_dataframe(  
    self,  
    df,  
    gt_lookup: dict, # accession → GroundTruth  
    accession_col: str = "ACCESSION_NUM",  
    pg_col: str = "PAIN_GENERATOR",  
    level_col: str = "LEVEL",  
    side_col: str = "SIDE",  
    nerve_col: str = "NERVE",  
    db_col: str = "DISC_OR_BONE_DISEASE",  
):
```

```
    """
```

Score an entire DataFrame of LLM outputs.

Returns the same DataFrame with PrIME score columns appended.

Missing accessions (not in gt_lookup) receive score 0 for all fields.

```
    """
```

```
    import pandas as pd
```

```
    records = []
```

```
    for _, row in df.iterrows():
```

```
        acc = row[accession_col]
```

```
gt = gt_lookup.get(acc)

if gt is None:

    # No GT available — mark as unscored

    records.append({

        "PrIME_PG": None, "PrIME_Side": None, "PrIME_Level": None,

        "PrIME_Nerve": None, "PrIME_DB": None,

        "PrIME_Composite_raw": None, "PrIME_Composite_pct": None,

    })

    continue
```

```
result = self.score_case(

    llm_pg = row.get(pg_col),

    llm_level = row.get(level_col),

    llm_side = row.get(side_col),

    llm_nerve = row.get(nerve_col),

    llm_db = row.get(db_col),

    gt = gt,

)

records.append(result.to_dict())
```

```
scored_cols = pd.DataFrame(records, index=df.index)

return pd.concat([df, scored_cols], axis=1)
```

```
#
```

```
# VALIDATION SUITE (run with: python prime_scoring.py)
```

```
#
```

```
def _run_tests():
```

```
    scorer = PRIMEScorer()
```

```
    failures = []
```

```
    def check(name, result, expected_pg, expected_side, expected_level,  
              expected_nerve, expected_db):
```

```
        actual = (result.pg, result.side, result.level, result.nerve, result.db)
```

```
        expected = (expected_pg, expected_side, expected_level, expected_nerve, expected_db)
```

```
        status = "PASS" if actual == expected else "FAIL"
```

```
        if status == "FAIL":
```

```
            failures.append(name)
```

```
            print(f" [{status}] {name}")
```

```
            print(f" got={actual} expected={expected}")
```

```
            if status == "FAIL":
```

```
                print(f" rationale: PG={result.pg_rationale} | "
```

```
                    f"Side={result.side_rationale} | Level={result.level_rationale} | "
```

```
                    f"Nerve={result.nerve_rationale} | DB={result.db_rationale}")
```

```
    print("=" * 60)
```

```
    print("PRIME SCORING VALIDATION SUITE")
```

```
    print("=" * 60)
```


— Test 1: Perfect consensus match —————

```
gt = build_gt(
yc_pg="Stenosis: Subarticular", fran_pg="Stenosis: Subarticular",
yc_level="L4-5", fran_level="L4-5",
yc_side="Right", fran_side="Right",
yc_nerve="L5", fran_nerve="L5",
yc_db="Disc", fran_db="Disc",
)
r = scorer.score_case("stenosis: subarticular", "L4-5", "right", "L5", "disc", gt)
check("Perfect consensus match", r, 4, 4, 4, 4, 4)
```

— Test 2: Non-consensus PG — LLM matches one expert —————

```
gt = build_gt(
yc_pg="Stenosis: Subarticular", fran_pg="Stenosis: Foraminal",
yc_level="L4-5", fran_level="L4-5",
yc_side="Right", fran_side="Right",
yc_nerve="L5", fran_nerve="S1",
yc_db="Disc", fran_db="Disc",
)
r = scorer.score_case("stenosis: subarticular", "L4-5", "right", "S1", "disc", gt)
check("Non-consensus PG: match one expert (score 3)", r, 3, 4, 4, 3, 4)
```

— Test 3: Same PG family, wrong subtype —————

```
gt = build_gt(
yc_pg="Stenosis: Canal", fran_pg="Stenosis: Canal",
yc_level="L4-5", fran_level="L4-5",
```

```
yc_side="Bilateral", fran_side="Bilateral",
yc_nerve=None, fran_nerve=None,
yc_db="Bone", fran_db="Bone",
)
r = scorer.score_case("stenosis: foraminal", "L4-5", "bilateral", "ns", "bone", gt)
check("Same PG family, wrong subtype (stenosis→stenosis, score 2)", r, 2, 4, 4, 4, 4)
```

— Test 4: Wrong disease family (disc vs bone) —————

```
gt = build_gt(
yc_pg="Facet Arthropathy", fran_pg="Facet Arthropathy",
yc_level="L4-5", fran_level="L4-5",
yc_side="Left", fran_side="Left",
yc_nerve=None, fran_nerve=None,
yc_db="Bone", fran_db="Bone",
)
r = scorer.score_case("disc hrn/annular tear", "L4-5", "left", "ns", "bone", gt)
check("Wrong PG family (disc vs bone_joint, score 1)", r, 1, 4, 4, 4, 4)
```

— Test 5: PG hallucination —————

```
gt = build_gt(
yc_pg="Stenosis: Subarticular", fran_pg="Stenosis: Subarticular",
yc_level="L4-5", fran_level="L4-5",
yc_side="Right", fran_side="Right",
yc_nerve="L5", fran_nerve="L5",
yc_db="Disc", fran_db="Disc",
)
```

```
r = scorer.score_case("pain generator", "L4-5", "right", "L5", "disc", gt)
check("PG hallucination — field name as answer (score 0)", r, 0, 4, 4, 4, 4)
```

```
# — Test 6: Level adjacent  $\pm 1$  from consensus —————
```

```
gt = build_gt(
yc_pg="Stenosis: Subarticular", fran_pg="Stenosis: Subarticular",
yc_level="L4-5", fran_level="L4-5",
yc_side="Right", fran_side="Right",
yc_nerve="L5", fran_nerve="L5",
yc_db="Disc", fran_db="Disc",
)
r = scorer.score_case("stenosis: subarticular", "L3-4", "right", "L5", "disc", gt)
check("Level  $\pm 1$  from consensus (score 3)", r, 4, 4, 3, 4, 4)
```

```
# — Test 7: Level  $\pm 2$  from consensus —————
```

```
r = scorer.score_case("stenosis: subarticular", "L2-3", "right", "L5", "disc", gt)
check("Level  $\pm 2$  from consensus (score 2)", r, 4, 4, 2, 4, 4)
```

```
# — Test 8: Cervical level (score 0) —————
```

```
r = scorer.score_case("stenosis: subarticular", "C4-5", "right", "L5", "disc", gt)
check("Cervical level in lumbar case (score 0)", r, 4, 4, 0, 4, 4)
```

```
# — Test 9: Side — bilateral hedging (score 2) —————
```

```
gt = build_gt(
yc_pg="Facet Arthropathy", fran_pg="Facet Arthropathy",
yc_level="L5-S1", fran_level="L5-S1",
```

```
yc_side="Left", fran_side="Left",
yc_nerve=None, fran_nerve=None,
yc_db="Bone", fran_db="Bone",
)
r = scorer.score_case("facet arthropathy", "L5-S1", "bilateral", "ns", "bone", gt)
check("Side hedged bilateral when experts said left (score 2)", r, 4, 2, 4, 4, 4)
```

```
# — Test 10: Side opposite (score 1) —————
r = scorer.score_case("facet arthropathy", "L5-S1", "right", "ns", "bone", gt)
check("Side opposite (right vs left, score 1)", r, 4, 1, 4, 4, 4)
```

```
# — Test 11: Nerve NS match —————
gt = build_gt(
yc_pg="Facet Arthropathy", fran_pg="Facet Arthropathy",
yc_level="L5-S1", fran_level="L5-S1",
yc_side="Bilateral", fran_side="Bilateral",
yc_nerve=None, fran_nerve=None,
yc_db="Bone", fran_db="Bone",
)
r = scorer.score_case("facet arthropathy", "L5-S1", "bilateral", "ns", "bone", gt)
check("Nerve NS: both model and experts say no nerve (score 4)", r, 4, 4, 4, 4, 4)
```

```
# — Test 12: Nerve named when expert says NS (score 2) —————
r = scorer.score_case("facet arthropathy", "L5-S1", "bilateral", "S1", "bone", gt)
check("Nerve named (S1) when experts said NS (score 2)", r, 4, 4, 4, 2, 4)
```

— Test 13: DB hedged "disc or bone" when consensus is disc (score 2) —

```
gt = build_gt(
yc_pg="Disc hrn/annular tear", fran_pg="Disc hrn/annular tear",
yc_level="L4-5", fran_level="L4-5",
yc_side="Left", fran_side="Left",
yc_nerve="L5", fran_nerve="L5",
yc_db="Disc", fran_db="Disc",
)
r = scorer.score_case("disc hrn/annular tear", "L4-5", "left", "L5", "disc or bone", gt)
check("DB hedged 'disc or bone' when expert said disc (score 2)", r, 4, 4, 4, 4, 2)
```

— Test 14: DB wrong (bone when disc) —————

```
r = scorer.score_case("disc hrn/annular tear", "L4-5", "left", "L5", "bone", gt)
check("DB wrong (bone vs disc, score 1)", r, 4, 4, 4, 4, 1)
```

— Test 15: Composite formula check —————

```
gt = build_gt(
yc_pg="Stenosis: Canal", fran_pg="Stenosis: Canal",
yc_level="L4-5", fran_level="L4-5",
yc_side="Bilateral", fran_side="Bilateral",
yc_nerve=None, fran_nerve=None,
yc_db="Bone", fran_db="Bone",
)
r = scorer.score_case("stenosis: canal", "L4-5", "bilateral", "ns", "bone", gt)
check("All 4s → composite = 100.0", r, 4, 4, 4, 4, 4)
comp_ok = abs(r.composite_pct - 100.0) < 0.01
```

```

print(f" [{'PASS' if comp_ok else 'FAIL'}] Composite pct = {r.composite_pct}")

if not comp_ok:

failures.append("Composite formula")


# — Test 16: Mistral hallucination example from Appendix Figure 1 —————
gt = build_gt(
yc_pg="Facet Arthropathy", fran_pg="Facet Arthropathy",
yc_level="L4-5", fran_level="L4-5",
yc_side="Bilateral", fran_side="Bilateral",
yc_nerve=None, fran_nerve=None,
yc_db="Bone", fran_db="Bone",
)

# Mistral output: "pain_generator" | "lumbar" | bilateral | "nerve" | disc
r = scorer.score_case("pain_generator", "lumbar", "bilateral", "nerve", "disc", gt)

# Expected: PG=0 (field name), Side=4 (bilateral=bilateral), Level=0 (region not level),
# Nerve=4 (both NS), DB=1 (disc vs bone — wrong but parseable)
check("Appendix Fig 1 Mistral hallucination example", r, 0, 4, 0, 4, 1)


print("=" * 60)

if failures:

print(f"FAILED TESTS ({len(failures)}): {failures}")

else:

print(f"All tests PASSED.")

print("=" * 60)

return len(failures) == 0

```

```
if __name__ == "__main__":
```

```
    _run_tests()
```